

بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-214 Computer Architecture and Organization

28 - Floating Point Operations

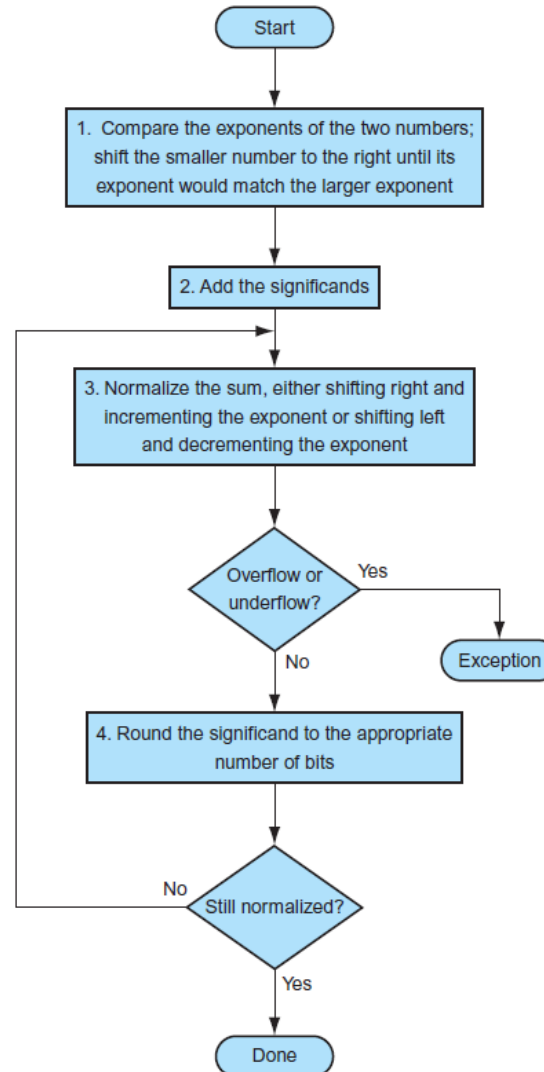
Introduction

- Floating-point addition
- Floating-point subtraction
- Floating-point multiplication
- Floating-point division

Floating-Point Addition

- Flowchart shows algorithm for binary FP addition
- Adjust significand of the number with the smaller exponent and then add the two significands
- Normalizes results, forcing a check for overflow or underflow
- Test for overflow and underflow depends on the precision of the operands

Addition Algorithm



Example

- Add the numbers 0.5_{ten} and 0.4375_{ten} in binary using the addition algorithm.
- Binary version of the two numbers in normalized scientific notation, assuming that we keep 4 bits of precision:

$$\begin{aligned} 0.5_{\text{ten}} &= 1/2_{\text{ten}} &= 1/2^1_{\text{ten}} &= 0.1_{\text{two}} \times 2^0 &= 1.000_{\text{two}} \times 2^{-1} \\ -0.4375_{\text{ten}} &= -7/16_{\text{ten}} &= -7/2^4_{\text{ten}} &= -0.0111_{\text{two}} \times 2^0 &= -1.110_{\text{two}} \times 2^{-2} \end{aligned}$$

Example (cont.)

- Step 1: Significand of number with the lesser exponent ($-1.11_{\text{two}} \times 2^2$) is shifted right until its exponent matches the larger number:

$$-1.110_{\text{two}} \times 2^2 = -0.111_{\text{two}} \times 2^{-1}$$

Example (cont.)

- Step 1: Significand of number with the lesser exponent ($-1.11_{\text{two}} \times 2^2$) is shifted right until its exponent matches the larger number:

$$-1.110_{\text{two}} \times 2^2 = -0.111_{\text{two}} \times 2^{-1}$$

- Step 2. Add the significands:

$$1.000_{\text{two}} \times 2^{-1} + (-0.111_{\text{two}} \times 2^{-1}) = 0.001_{\text{two}} \times 2^{-1}$$

Example (cont.)

- Step 3. Normalize the sum, checking for overflow or underflow:

$$\begin{aligned}0.001_{\text{two}} \times 2^{-1} &= 0.010_{\text{two}} \times 2^{-2} = 0.100_{\text{two}} \times 2^{-3} \\ &= 1.000_{\text{two}} \times 2^{-4}\end{aligned}$$

- Since $127 \geq +4 \geq -126$, there is no overflow or underflow
 - Biased exponent would be $-4 + 127$, or 123, which is between 1 and 254, the smallest and largest unreserved biased exponents.

Example (cont.)

- Step 4: Round the sum:

$$1.000_{\text{two}} \times 2^{-4}$$

- Sum fits exactly in 4 bits, so no change to bits due to rounding
- Sum is then

$$\begin{aligned} 1.000_{\text{two}} \times 2^{-4} &= 0.0001000_{\text{two}} = 0.0001_{\text{two}} \\ &= 1/2^4_{\text{ten}} = 1/16_{\text{ten}} = 0.0625_{\text{ten}} \end{aligned}$$

- Sum is what we would expect from adding 0.5_{ten} to 0.4375_{ten}

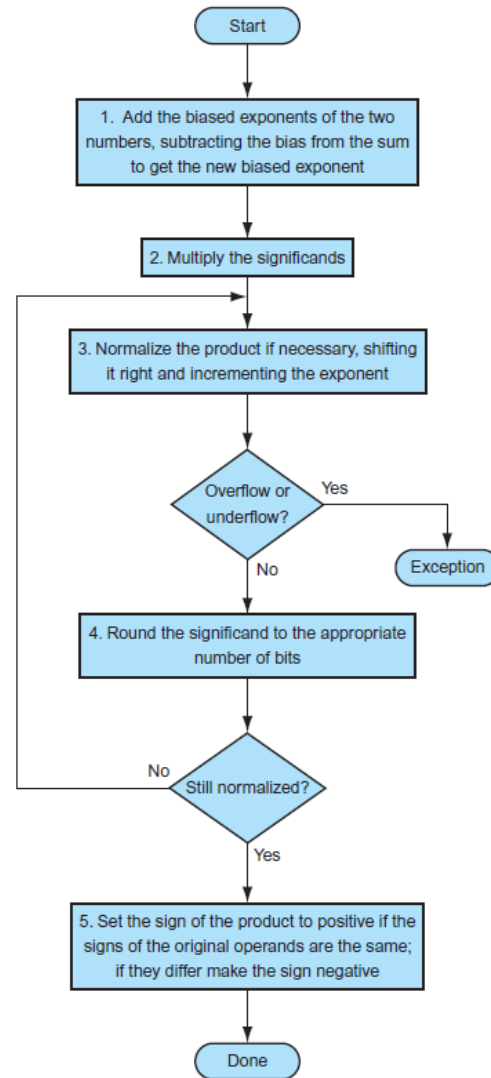
Floating-Point Subtraction

- Similar to addition except in
 - Step 2: subtract significands

Floating-Point Multiplication

- Flowchart shows multiplication of binary FP numbers
- Calculating new exponent of product by adding biased exponents, being sure to subtract one bias to get proper result
- Multiplication of significands, followed by optional normalization
- Size of exponent is checked for overflow or underflow, and then product is rounded
 - If rounding leads to further normalization, again check for exponent size
- Set sign bit to 1 if signs of operands were different (negative product) or to 0 if they were same (positive product)

Multiplication Algorithm



Example

- Multiply the numbers 0.5_{ten} and 0.4375_{ten} .

Example

- Multiply the numbers 0.5_{ten} and 0.4375_{ten} .
- In binary, task is multiplying $1.000_{\text{two}} \times 2^1$ by $1.110_{\text{two}} \times 2^2$.

– Step 1. Adding exponents without bias:

$$-1 + (-2) = -3$$

- Using biased representation:

$$\begin{aligned} (-1 + 127) + (-2 + 127) - 127 &= (-1 - 2) + (127 + 127 - 127) \\ &= -3 + 127 = 124 \end{aligned}$$

Example (cont.)

- Step 2. Multiplying significands:

$$\begin{array}{r} \times \quad 1.000_{\text{two}} \\ \quad 1.110_{\text{two}} \\ \hline \quad 0000 \\ \quad 1000 \\ \quad 1000 \\ \quad 1000 \\ \hline 1110000_{\text{two}} \end{array}$$

Example (cont.)

– Step 2. Multiplying significands:

$$\begin{array}{r} 1.000_{\text{two}} \\ \times 1.110_{\text{two}} \\ \hline 0000 \\ 1000 \\ 1000 \\ 1000 \\ \hline 1110000_{\text{two}} \end{array}$$

- Product is $1.110000_{\text{two}} \times 2^{-3}$
- Need to keep it to 4 bits, so it is $1.110_{\text{two}} \times 2^{-3}$

Example (cont.)

- Step 3. Check product to make sure it is normalized, and then check exponent for overflow or underflow
 - Product is already normalized
 - Since $127 \geq 3 \geq -126$, there is no overflow or underflow. (Using biased representation, $254 \geq 124 \geq 1$, exponent fits.)

Example (cont.)

- Step 3. Check product to make sure it is normalized, and then check exponent for overflow or underflow
 - Product is already normalized
 - Since $127 \geq 3 \geq -126$, there is no overflow or underflow. (Using biased representation, $254 \geq 124 \geq 1$, exponent fits.)
- Step 4. Rounding product makes no change:

$$1.110_{\text{two}} \times 2^{-3}$$

Example (cont.)

- Step 3. Check product to make sure it is normalized, and then check exponent for overflow or underflow
 - Product is already normalized
 - Since $127 \geq 3 \geq -126$, there is no overflow or underflow. (Using biased representation, $254 \geq 124 \geq 1$, exponent fits.)
- Step 4. Rounding product makes no change:

$$1.110_{\text{two}} \times 2^{-3}$$

- Step 5. Since signs of original operands differ, product sign is negative
 - Hence, product is

$$-1.110_{\text{two}} \times 2^{-3}$$

Floating-Point Division

- Similar to multiplication except in
 - Step 1: subtract exponents
 - Step 2: divide significands